



دانشگاه قم

دانشکده فنی و مهندسی

کارآموزی کارشناسی رشته مهندسی کامپیوتر

عنوان:

طراحی و پیاده‌سازی سیستم بینایی ماشین و پردازش زنده مارکرهای
بینایی ArUco در محیط اندروید با استفاده از کتابخانه OpenCV
همراه با مهارت‌های عملی شبکه و امنیت

استاد کارآموزی:

دکتر محسن نیک رای

نگارنده:

امید شعبانعلی

شهریور ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

تقدیر:

از زحمات استاد محترم کارآموزی جناب آقای دکتر نیک رای و سرپرست محترم کارآموزی اینجانب
جناب آقای جاویدی تشکر و قدردانی می نمایم.

چکیده:

دوره کارآموزی حاضر با هدف تلفیق دانش نظری مهندسی کامپیوتر با کاربردهای عملی پیشرفته در دو حوزه بینایی ماشین در سیستم‌های همراه (**Mobile Computer Vision**) و زیرساخت و امنیت شبکه پرداخته شده است. بخش نرم‌افزاری و هسته اصلی پروژه شامل طراحی و توسعه یک اپلیکیشن هوشمند اندرویدی به زبان Kotlin تحت عنوان **ArucoDetector** است که توانایی استخراج، پردازش، و شناسایی زمان‌واقعی (Real-Time) مارک‌های بینایی **ArUco** را با پردازش تصویر پیشرفته از طریق دوربین دستگاه دارد. در این پروژه با غلبه بر چالش‌های پیچیده مدیریت حافظه **Native C++** در قالب **Java Native Interface (JNI)**، منطق چنددیکشنری **ArUco**، بهینه‌سازی نسبت تصویر (**Aspect Ratio**) و رفع خطاهای پیچیده سیستم ساخت **Gradle**، یک سیستم پایدار و با کارایی بالا ارائه شده است.

علاوه بر بخش نرم‌افزاری، در طول این دوره مهارت‌های عملی ارزشمندی در حوزه پیکربندی و نگهداری شبکه‌های کامپیوتر از جمله شبیه‌سازی شبکه‌ها با **Cisco Packet Tracer**، آماده‌سازی سخت‌افزاری کابل‌های شبکه سوکت‌زنی (**RJ-45**)، اتصال پریزهای شبکه (کیستون‌زنی) و مباحث بنیادی امنیت شبکه کسب گردیده که در فصل نهایی این گزارش به‌طور کامل تبیین شده‌اند.

فهرست مطالب

۱	فصل ۱: کلیات و معرفی دوره کارآموزی
۱-۱-۱	مقدمه
۱-۲-۱	اهداف دوره کارآموزی
۳	فصل ۲ مبانی علمی و نظری پروژه
۳	(ArUco & OpenCV)
۲-۱-۲	مارکرهای ArUco چیستند؟
۲-۲-۲	الگوریتم شناسایی ArUco در OpenCV
۲-۳-۲	معماری OpenCV در اندروید
۶	فصل ۳: معماری و پیاده‌سازی فنی نرم‌افزار ArucoDetector
در این فصل، ساختار پروژه، پیکربندی فایل‌های گرادل، لایه مانیفست، واژه نامه نسخه‌ها، فایل‌های XML و کد اصلی به تفصیل بررسی می‌شوند.	
۳-۱-۳	ساختار فایل‌های پروژه و تنظیمات Gradle
۳-۲-۳	پیکربندی مانیفست و رابط کاربری (UI)
۳-۳-۳	هسته اصلی پردازش تصویر MainActivity.kt :
۲۱	فصل ۴: تحلیل چالش‌های فنی و حل آن‌ها
۴-۱-۴	چالش کشیدگی تصویر (Aspect Ratio Distortion)
۴-۲-۴	چالش نشت حافظه در C++ Native Heap (Memory Leak)
۴-۳-۴	چالش تعارض کتابخانه‌ها و پیکربندی AndroidX
۲۴	فصل ۵: دستاوردها و مهارت‌های افزوده در حوزه شبکه و امنیت
یکی از بخش‌های مکمل و کاربردی دوره کارآموزی، فراگیری مهارت‌های پس‌یو (سخت‌افزاری) و اکتیو (نرم‌افزاری) شبکه بوده است. در این بخش گزارش مفصلی از آموزش‌های دریافتی ارائه می‌گردد:	
۵-۱-۵	آشنایی و کار با نرم‌افزار Cisco Packet Tracer
۵-۲-۵	مهارت عملی سوکت زدن کابل شبکه (Crimping RJ-۴۵)
۵-۳-۵	مهارت کیستون زدن پریز شبکه (Keystone Jack Punch Down)
۵-۴-۵	مباحث کلی و کاربردی امنیت شبکه
۲۹	فصل ۶: نتیجه‌گیری و پیشنهادات
۶-۱-۶	نتیجه‌گیری

۶-۲- پیشنهادات برای توسعه آتی ۳۰

فصل ۱:

کلیات و معرفی دوره کارآموزی

۱-۱- مقدمه

با رشد روزافزون پردازنده‌های چندهسته‌ای و واحدهای پردازش گرافیکی روی تراشه‌های همراه، پردازش تصویر در زمان واقعی روی گوشی‌های هوشمند به یکی از جذاب‌ترین و کاربردی‌ترین زمینه‌های مهندسی کامپیوتر تبدیل شده است. مارکرهای بینایی ماشین (Fiducial Markers)، به‌ویژه مارکرهای ArUco، نقش کلیدی در سیستم‌های واقعیت افزوده (AR)، هدایت ربات‌های خودگردان، انبارداری هوشمند و موقعیت‌یابی داخلی (Indoor Positioning) ایفا می‌کنند.

هدف اولیه این پروژه، پیاده‌سازی یک اپلیکیشن اندرویدی یکپارچه بود که بتواند بدون افت نرخ فریم (FPS) و بدون داغ شدن یا مصرف بی‌رویه منابع دستگاه، فریم‌های پردازشی را از دوربین دریافت کرده و پس از اعمال الگوریتم‌های آستانه گذاری تطبیقی (Adaptive Thresholding) و تحلیل چندضلعی‌ها، مارکرهای موجود در محیط را شناسایی نموده و اطلاعات شناسه (ID) و مرزهای هندسی آن‌ها را روی صفحه به تصویر بکشد.

۱-۲- اهداف دوره کارآموزی

- تسلط بر کتابخانه OpenCV در اندروید: استفاده از نسخه ۴.۹.۰ کتابخانه OpenCV و لود کردن کتابخانه‌های نیتیو C++ در محیط کاتلین.
- پردازش زمان واقعی (Real-Time Processing): مدیریت چرخه حیات فریم‌ها در پردازش تصویر زنده بدون بروز وقفه (Lag) یا نشت حافظه (Memory Leak).
- پشتیبانی از دیکشنری‌های متعدد: شناسایی همزمان کدهای ArUco از استانداردهای مختلف مانند DICT_4X4_100 و DICT_6X6_250.
- بهینه‌سازی رابط کاربری و نمایش: جلوگیری از کشیدگی تصویر (Stretching) با تنظیم ابعاد فریم بر اساس استاندارد ۱۶:۹ HD.
- کسب مهارت‌های پسیو و اکتیو شبکه: یادگیری عملی کابل کشی، سوکت زنی، کیستون زنی، شبیه‌سازی شبکه و مفاهیم امنیت شبکه.

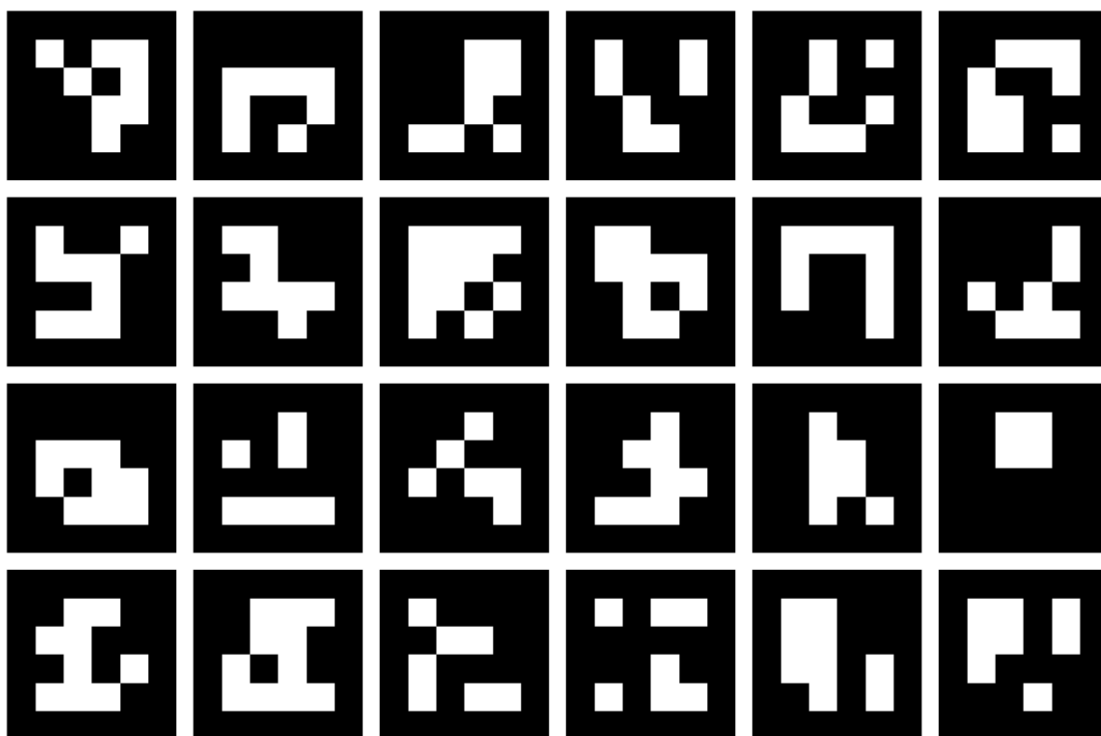
فصل ۲:

مبانی علمی و نظری پروژه

(ArUco & OpenCV)

۲-۱- مارک‌های ArUco چیستند؟

مارکر ArUco یک مربع دوبعدی ماتریسی است که شامل یک حاشیه سیاه بیرونی و یک ماتریس باینری (سیاه و سفید) داخلی است. حاشیه بیرونی به الگوریتم پردازش تصویر کمک می‌کند تا فرم هندسی مربع را در تصویر تشخیص دهد و ماتریس داخلی شامل کدهای تصحیح خطا (Error Correction Codes) بر پایه فاصله همینگ (Hamming Distance) است که شناسه منحصر به فرد مارکر را تعیین می‌کند.



تصویر ۱: ساختار هندسی و الگوی شطرنجی مارکر بینایی ArUco با خطوط راهنما و گوشه‌های شناسایی شده

تفاوت ArUco با QR Code :

۱. سرعت پردازش بالا: به دلیل ماتریس کوچک‌تر (مثلاً 4×4 یا 6×6 پیکسل باینری)، پردازش ArUco بسیار سریع‌تر از QR Code است.
۲. محاسبه ژست و زاویه: (Pose Estimation) به دلیل داشتن ۴ گوشه کاملاً مشخص و ابعاد فیزیکی معین، می‌توان ماتریس دوران (Rotation) و انتقال (Translation) دوربین نسبت به

مارکر را به دست آورد.

۲-۲- الگوریتم شناسایی ArUco در OpenCV

روند شناسایی مارکرهای ArUco شامل ۴ گام اصلی زیر است:

۱. آستانه‌گذاری تطبیقی: (Adaptive Thresholding) تبدیل تصویر رنگی به تصویر باینری (سیاه و سفید مطلق) جهت جداسازی حاشیه‌ها تحت شرایط نوری متغیر.
۲. استخراج کانتورها: (Contour Extraction) یافتن چندضلعی‌های ۴ ضلعی که کاندیدای مارکر هستند.
۳. تبدیل پرسپکتیو: (Perspective Transformation) صاف کردن تصویر کاندیدا و تبدیل آن به یک ماتریس مربع منظم.
۴. رمزگشایی ماتریس داخلی: (Matrix Decoding) خواندن سلول‌های سیاه و سفید و مقایسه آن با الگوی دیکشنری مرجع ($DICT_{4 \times 4_{100}}$ یا $DICT_{6 \times 6_{250}}$) و اعمال ضریب تصحیح خطا.

۲-۳- معماری OpenCV در اندروید

کتابخانه OpenCV یک کتابخانه نوشته‌شده به زبان C++ است. برای استفاده از آن در اندروید از **JNI (Java Native Interface)** استفاده می‌شود. کلاس `CameraBridgeViewBase` و مشتقات آن مانند `JavaCameraView` ارتباط مستقیمی بین لایه اندروید و موتور نیتیو OpenCV برقرار می‌کنند تا فریم‌ها به صورت ساختار `Mat` (ماتریس تصویر OpenCV) در اختیار کاتلین قرار گیرند.

فصل ۳:

معماری و پیاده‌سازی فنی نرم‌افزار

ArucoDetector

در این فصل، ساختار پروژه، پیکربندی فایل‌های گرادل، لایه مانیفست، واژه نامه نسخه‌ها، فایل‌های XML و کد اصلی به تفصیل بررسی می‌شوند.

۳-۱ - ساختار فایل‌های پروژه و تنظیمات Gradle

۱. فایل build.gradle.kts (سطح مازول app)

این فایل پیکربندی کامپایل و کتابخانه‌های وابسته را مشخص می‌کند. مطابق کد زیر، از OpenCV نسخه ۴.۹.۰ استفاده شده است:

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
}  
  
android {  
    namespace = "com.example.arucodetector"  
    compileSdk = 34  
  
    defaultConfig {  
        applicationId = "com.example.arucodetector"  
        minSdk = 24  
        targetSdk = 34  
        versionCode = 1  
        versionName = "1.0"  
  
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  
    }  
  
    buildTypes {  
        release {  
            isMinifyEnabled = false  
            proguardFiles(  
                getDefaultProguardFile("proguard-android-optimize.txt"),  

```

```

        "proguard-rules.pro"
    )
}
}
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_1_8
    targetCompatibility = JavaVersion.VERSION_1_8
}
kotlinOptions {
    jvmTarget = "1.8"
}
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.constraintlayout)
    implementation("org.opencv:opencv:4.9.0")
    implementation(fileTree(mapOf("dir" to "libs", "include" to listOf("*.jar"))))

    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
}

```

۲. فایل **libs.versions.toml (Version Catalog)**

مدیریت متمرکز نسخه‌های افزونه‌ها و کتابخانه‌ها در این فایل صورت گرفته است :

```

[versions]
agp = "8.7.0"
kotlin = "2.0.21"
coreKtx = "1.12.0"
junit = "4.13.2"
junitVersion = "1.1.5"

```

```

espressoCore = "3.5.1"
appcompat = "1.6.1"
material = "1.11.0"
activity = "1.8.2"
constraintlayout = "2.1.4"

```

```
[libraries]
```

```

androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core",
version.ref = "espressoCore" }
androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref =
"appcompat" }
material = { group = "com.google.android.material", name = "material", version.ref =
"material" }
androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity" }
androidx-constraintlayout = { group = "androidx.constraintlayout", name =
"constraintlayout", version.ref = "constraintlayout" }

```

```
[plugins]
```

```

android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }

```

۳. فایل gradle.properties

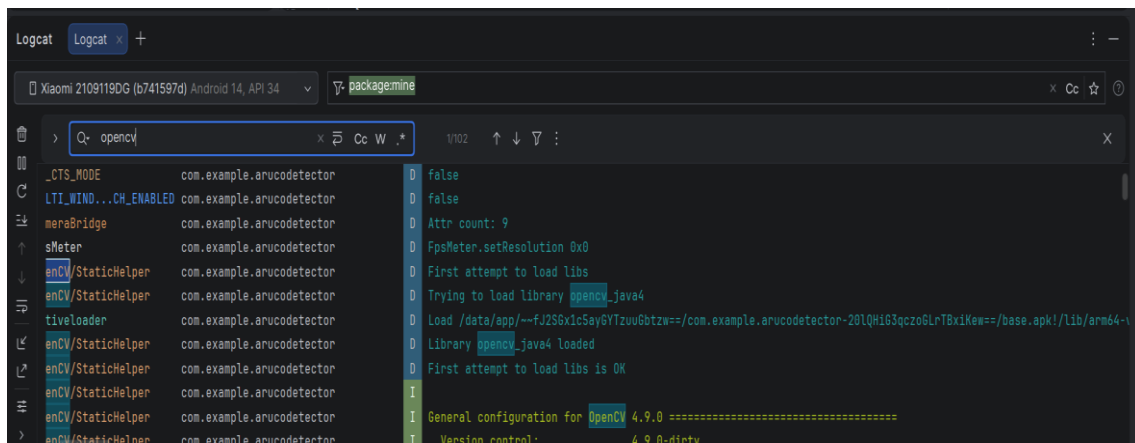
جهت سازگاری کامل با AndroidX و بهینه‌سازی حافظه اجرای گرادل (Daemon JVM) :

```

# Project-wide Gradle settings.
# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.
# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html
# Specifies the JVM arguments used for the daemon process.

```

```
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. For more details, visit
# https://developer.android.com/r/tools/gradle-multi-project-decoupled-projects
# org.gradle.parallel=true
# When enabled, the Configuration Cache allows Gradle to skip the configuration
# phase entirely if nothing that affects the build configuration (such as build scripts)
# has changed. Additionally, Gradle applies performance optimizations to task execution.
org.gradle.configuration-cache=true
# Kotlin code style for this project: "official" or "obsolete":
kotlin.code.style=official
android.useAndroidX=true
android.enableJetifier=true
```



تصویر ۲: محیط توسعه Android Studio و تایید بارگذاری موفق کتابخانه OpenCV در پنجره Logcat

۳-۲- پیکربندی مانیفست و رابط کاربری (UI)

۱. فایل AndroidManifest.xml

برای دسترسی به دوربین و تنظیم جهت نمایش به صورت افقی (Landscape) جهت تطابق بهتر با زوایای پردازش بینایی ماشین:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.arucodetector">

    <uses-permission android:name="android.permission.CAMERA" />
    <uses-feature android:name="android.hardware.camera" android:required="true" />
    <uses-feature
        android:name="android.hardware.camera.autofocus"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ArucoDetector"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="landscape"
            android:configChanges="orientation|screenSize|keyboardHidden">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
```

</manifest>

۲. فایل activity_main.xml

استفاده از `JavaCamera2View` جهت بهره‌گیری از `API` دوم دوربین اندروید (`API ۲ Camera`) برای لود فریم‌ها با سرعت و کیفیت بالاتر :

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<org.opencv.android.JavaCamera2View
    android:id="@+id/camera_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:camera_id="any"
    app:show_fps="true"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

۳-۳- هسته اصلی پردازش تصویر MainActivity.kt :

کد سورس کامل اکتیویته اصلی بدون هیچ‌گونه کامنت فارسی :

```
package com.example.arucodetector
```

```
import android.Manifest
```

```
import android.content.pm.PackageManager
```

```
import android.os.Bundle
import android.util.Log
import android.view.SurfaceView
import android.view.WindowManager
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import org.opencv.android.CameraBridgeViewBase
import org.opencv.android.CameraBridgeViewBase.CvCameraViewFrame
import org.opencv.android.CameraBridgeViewBase.CvCameraViewListener2
import org.opencv.android.OpenCVLoader
import org.opencv.core.*
import org.opencv.imgproc.Imgproc
import org.opencv.objdetect.ArucoDetector
import org.opencv.objdetect.DetectorParameters
import org.opencv.objdetect.Objdetect
import java.util.ArrayList

class MainActivity : AppCompatActivity(), CvCameraViewListener2 {

    private companion object {
        private const val TAG = "ArUcoDetector"
        private const val CAMERA_PERMISSION_REQUEST_CODE = 101
    }

    private lateinit var cameraView: CameraBridgeViewBase
    private var detectors: List<Pair<String, ArucoDetector>> = emptyList()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
        setContentView(R.layout.activity_main)
```

```
cameraView = findViewById(R.id.camera_view)
cameraView.visibility = SurfaceView.VISIBLE
cameraView.setCvCameraViewListener(this)
cameraView.setCameraIndex(CameraBridgeViewBase.CAMERA_ID_BACK)

// ASPECT RATIO OPTIMIZATION: Use 16:9 ratio to prevent image stretching on
modern devices
// Standard HD 1280x720 is 16:9
cameraView.setMaxFrameSize(1280, 720)

if (!checkCameraPermission()) {
    requestCameraPermission()
}

}

override fun onResume() {
    super.onResume()
    if (checkCameraPermission()) {
        startOpenCvCamera()
    }
}

private fun startOpenCvCamera() {
    if (OpenCVLoader.initLocal()) {
        Log.d(TAG, "OpenCV initialized successfully")
        cameraView.post {
            cameraView.setCameraPermissionGranted()
            cameraView.enableView()
        }
    } else {
        Log.e(TAG, "OpenCV initialization failed")
    }
}
```

```
}

override fun onPause() {
    super.onPause()
    cameraView.disableView()
}

override fun onDestroy() {
    super.onDestroy()
    cameraView.disableView()
}

private fun checkCameraPermission(): Boolean {
    return ContextCompat.checkSelfPermission(
        this,
        Manifest.permission.CAMERA
    ) == PackageManager.PERMISSION_GRANTED
}

private fun requestCameraPermission() {
    ActivityCompat.requestPermissions(
        this,
        arrayOf(Manifest.permission.CAMERA),
        CAMERA_PERMISSION_REQUEST_CODE
    )
}

override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<out String>,
    grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
```

```

        if (requestCode == CAMERA_PERMISSION_REQUEST_CODE &&
grantResults.isNotEmpty() && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
            startOpenCvCamera()
        } else {
            Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show()
        }
    }
}

```

```

override fun onCameraViewStarted(width: Int, height: Int) {
    try {
        val parameters = DetectorParameters()
        // Optimized parameters for mobile handheld use
        parameters.set_minMarkerPerimeterRate(0.03)
        parameters.set_adaptiveThreshConstant(7.0)
        parameters.set_errorCorrectionRate(0.6)

        // Support both 4x4 (notebook) and 6x6 (user's other markers)
        detectors = listOf(
            "4X4_100" to
ArucoDetector(Objdetect.getPredefinedDictionary(Objdetect.DICT_4X4_100), parameters),
            "6X6_250" to
ArucoDetector(Objdetect.getPredefinedDictionary(Objdetect.DICT_6X6_250), parameters)
        )

        Log.d(TAG, "Detectors initialized: 4x4 and 6x6")
    } catch (e: Exception) {
        Log.e(TAG, "Error initializing ArUco Detectors: ${e.message}")
    }
}

```

```

override fun onCameraViewStopped() {
    // Class-level Mat/List were removed as we use local ones for memory safety
}

```

```
}
```

```
override fun onCameraFrame(inputFrame: CvCameraViewFrame): Mat {
```

```
    val rgba = inputFrame.rgba()
```

```
    val gray = inputFrame.gray()
```

```
    var detectedAny = false
```

```
    var detectedDictName = ""
```

```
    for ((name, detector) in detectors) {
```

```
        val currentCorners = ArrayList<Mat>()
```

```
        val currentIds = Mat()
```

```
        try {
```

```
            detector.detectMarkers(gray, currentCorners, currentIds)
```

```
            if (!currentIds.empty()) {
```

```
                detectedAny = true
```

```
                detectedDictName = name
```

```
                // Display IDs and Dictionary Name
```

```
                for (i in 0 until currentCorners.size) {
```

```
                    val id = currentIds.get(i, 0)[0].toInt()
```

```
                    val corner = currentCorners[i]
```

```
                    // Extract 4 corners from Mat (1x4 CV_32FC2)
```

```
                    val p0 = Point(corner.get(0, 0)[0], corner.get(0, 0)[1])
```

```
                    val p1 = Point(corner.get(0, 1)[0], corner.get(0, 1)[1])
```

```
                    val p2 = Point(corner.get(0, 2)[0], corner.get(0, 2)[1])
```

```
                    val p3 = Point(corner.get(0, 3)[0], corner.get(0, 3)[1])
```

```
                    // 1. Draw GREEN Box (Thick lines for visibility)
```

```
                    val green = Scalar(0.0, 255.0, 0.0, 255.0)
```

```

    Imgproc.line(rgba, p0, p1, green, 4)
    Imgproc.line(rgba, p1, p2, green, 4)
    Imgproc.line(rgba, p2, p3, green, 4)
    Imgproc.line(rgba, p3, p0, green, 4)

    // 2. Draw RED ID Number in Center
    val centerX = (p0.x + p1.x + p2.x + p3.x) / 4.0
    val centerY = (p0.y + p1.y + p2.y + p3.y) / 4.0
    Imgproc.putText(
        rgba, "$id", Point(centerX - 15, centerY + 15),
        Imgproc.FONT_HERSHEY_SIMPLEX, 1.0, Scalar(255.0, 0.0, 0.0, 255.0), 3
    )

    // 3. Draw BLUE ID Label
    Imgproc.putText(
        rgba, "ID: $id", Point(p0.x, p0.y - 20),
        Imgproc.FONT_HERSHEY_SIMPLEX, 1.0, Scalar(0.0, 0.0, 255.0, 255.0), 3
    )
}
}
} catch (e: Exception) {
    Log.e(TAG, "Detection error ($name): ${e.message}")
} finally {
    currentIds.release()
    for (mat in currentCorners) mat.release()
}
}

// Status Feedback
val statusText = if (detectedAny) "ArUco Detected: $detectedDictName" else "No ArUco
Detected"

val statusColor = if (detectedAny) Scalar(0.0, 255.0, 0.0, 255.0) else Scalar(255.0, 255.0,
255.0, 255.0)

```

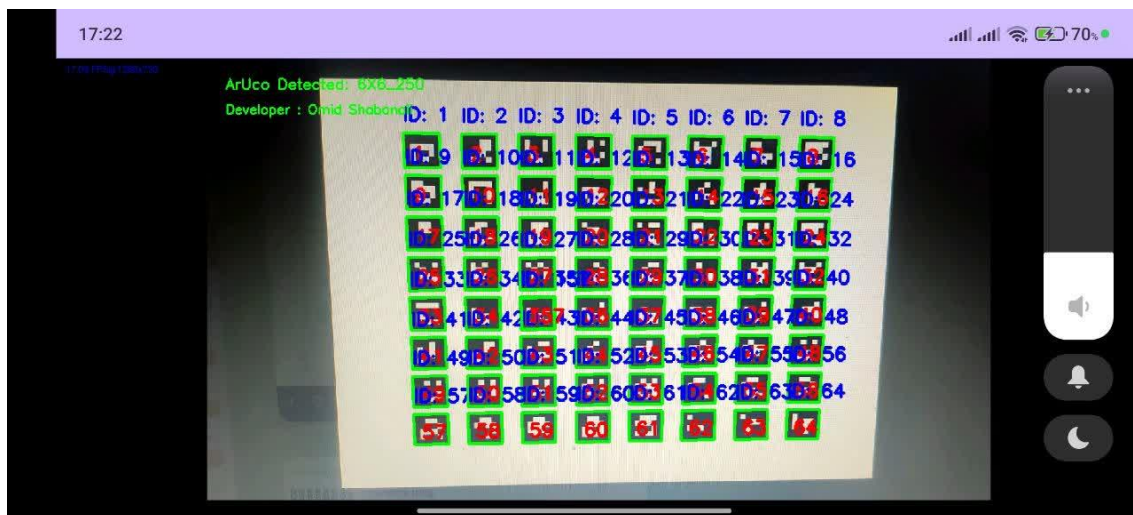
```

// Line 1: Status
Imgproc.putText(
    rgba, statusText, Point(30.0, 50.0),
    Imgproc.FONT_HERSHEY_SIMPLEX, 0.8, statusColor, 2
)

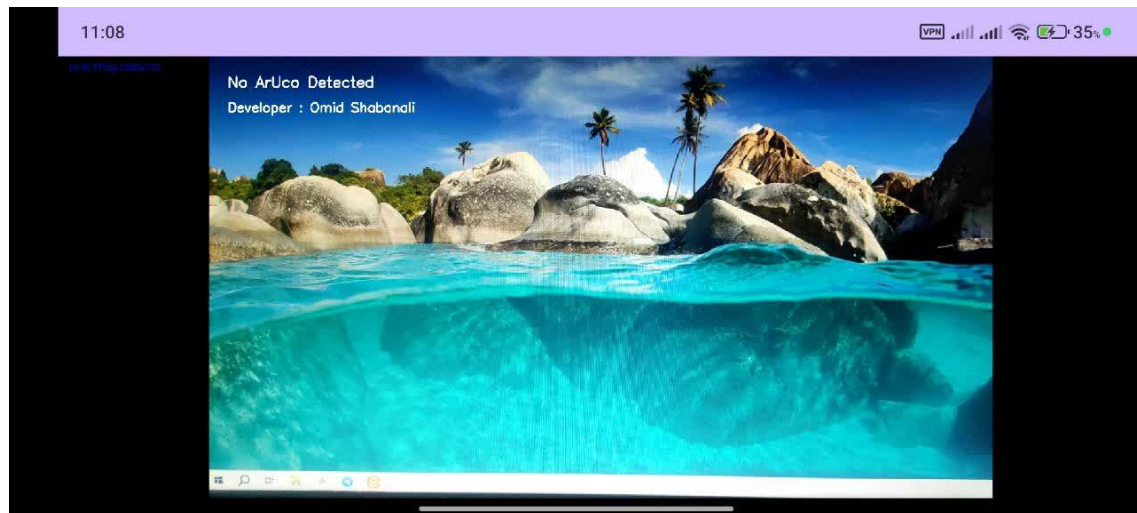
// Line 2: Developer Info
Imgproc.putText(
    rgba, "Developer : Omid Shabanali", Point(30.0, 90.0),
    Imgproc.FONT_HERSHEY_SIMPLEX, 0.7, statusColor, 2
)

return rgba
}
}

```



تصویر ۳: نمای کارکرد زنده برنامه ArucoDetector روی دستگاه اندرویدی؛ نمایش مربع سبز دور مارکر، شناسه قرمز در مرکز و مشخصات توسعه‌دهنده در بالای صفحه



تصویر ۴: وضعیت برنامه در حالت عدم شناسایی مارکر نمایش متن سفید (No ArUco Detected)

فصل ۴:

تحلیل چالش‌های فنی و حل آن‌ها

در طول توسعه پروژه نرم‌افزاری با چند چالش مهندسی اساسی روبرو شدیم که نحوه عیب‌یابی و برطرف‌سازی آن‌ها به شرح زیر است:

۱-۴ - چالش کشیدگی تصویر (Aspect Ratio Distortion)

مشکل: دوربین‌های مدرن اندروید دارای نسبت ابعاد ۱۶:۹ یا ۲۰:۹ هستند، در حالی که الگوریتم‌های پیش‌فرض OpenCV فریم‌ها را با نسبت ۴:۳ دریافت می‌کردند. این عدم تطابق باعث کشیده شدن مربع‌های ArUco به مستطیل شده و الگوریتم تشخیص گوشه با خطا مواجه می‌شد.

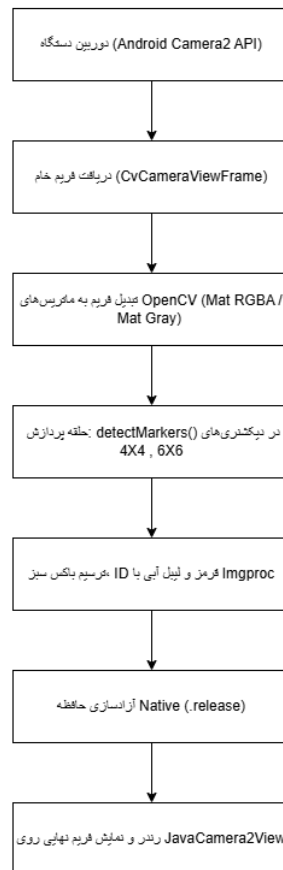
راهکار: با اعمال `cameraView.setMaxFrameSize(۱۲۸۰, ۷۲۰)` ابعاد فریم به‌طور ثابت روی رزولوشن standard HD (۱۶:۹) قفل شد و مشکل دفرمه شدن هندسی کاملاً برطرف گردید.

۲-۴ - چالش نشت حافظه در (C++ Native Heap Memory Leak)

- مشکل:** در پردازش ۳۰ فریم در ثانیه، تولید شیء‌های Mat بدون آزادسازی باعث پُر شدن سریع حافظه Native RAM و کرش ناگهانی برنامه با خطای OutOfMemoryError می‌شد.
- راهکار:** مطابق کد بالا، تمام شیء‌های `currentIds` و `currentCorners` در بلوک `finally` صریحاً با متد `release()` از حافظه RAM نیتو پاکسازی شدند.

۳-۴ - چالش تعارض کتابخانه‌ها و پیکربندی AndroidX

- مشکل:** با بروزرسانی Gradle به نسخه ۸.۷ و کاتلین ۲.۰.۲۱، خطاهای مربوط به عدم شناسایی ویژگی‌های AndroidX رخ داد.
- راهکار:** دو دستور `android.useAndroidX=true` و `android.enableJetifier=true` به فایل `gradle.properties` اضافه شد تا کدهای قدیمی به استانداردهای جدید ترانسپایل شوند.



تصویر ۵: دیاگرام جریان داده (Data Flow Diagram) و چرخه حیات فریم‌ها در OpenCV Android از دوربین تا رندر روی نمایشگر

فصل ۵:

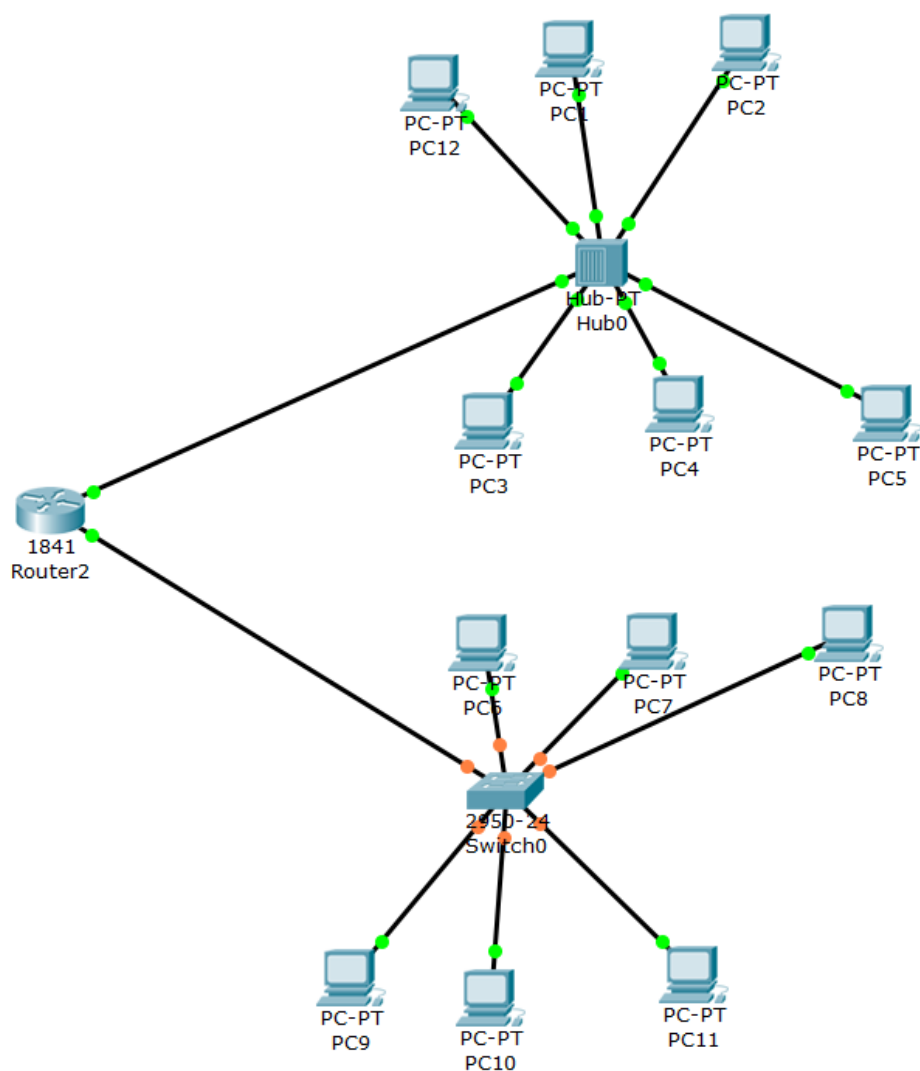
دستاوردها و مهارت‌های افزوده در حوزه

شبکه و امنیت

یکی از بخش‌های مکمل و کاربردی دوره کارآموزی، فراگیری مهارت‌های پس‌یو (سخت‌افزاری) و اکتیو (نرم‌افزاری) شبکه بوده است. در این بخش گزارش مفصلی از آموزش‌های دریافتی ارائه می‌گردد:

۵-۱- آشنایی و کار با نرم‌افزار Cisco Packet Tracer

- در این بخش، شبیه‌سازی شبکه‌های کامپیوتر محلی (LAN) و گسترده (WAN) تمرین شد:
- **طراحی توپولوژی‌های شبکه:** پیاده‌سازی توپولوژی‌های Star (ستاره‌ای) و Mesh با استفاده از سویچ‌های Cisco ۲۹۶۰ و روترهای سری ۱۹۴۱.
- **پیکربندی آدرس‌دهی IP:** تخصیص آدرس‌های IPv۴ به صورت Static و کانفیگ سرویس DHCP Server بر روی روتر جهت تخصیص خودکار آدرس به کلاینت‌ها.
- **ایجاد شبکه‌های مجازی (VLAN):** دسته‌بندی پورت‌های سویچ در VLAN‌های مجزا (مانند ۱۰ و ۲۰ VLAN) جهت ایزوله‌سازی ترافیک بخش‌های مختلف و تنظیم Trunking با پروتکل Q.۸۰۲.۱
- **تست اتصال:** استفاده از دستورات ping و traceroute جهت عیب‌یابی مسیرهای عبور بسته و تحلیل خطاهای ARP و ICMP.



تصویر ۶: شبیه‌سازی توپولوژی شبکه شامل روتر ۱۸۴۱، سوئیچ، هاب و تفکیک زیرشبکه‌ها در Cisco Packet Tracer

۲-۵- مهارت عملی سوکت زدن کابل شبکه (Crimping RJ-۴۵)

آمادگی برای کابل کشی ساخت یافته نیازمند ساخت کابل‌های patch cord استاندارد است:

- استانداردهای رنگ‌بندی: فراگیری استانداردهای T568A و T568B (استاندارد رایج).

○ ترتیب رنگ استاندارد: T568B

۱. سفید نارنجی
۲. نارنجی
۳. سفید سبز

۴. آبی

۵. سفید آبی

۶. سبز

۷. سفید قهوه‌ای

۸. قهوه‌ای

• مراحل انجام کار:

۱. لخت کردن روکش بیرونی کابل شبکه (Cat6 UTP) با ابزار سیم‌لخت کن (Stripping Tool).

۲. مرتب‌سازی و صاف کردن زوج‌سیم‌ها طبق استاندارد T568B.

۳. برش یک‌دست انتهای سیم‌ها با قیچی آچار سوکت‌زن (Crimper).

۴. وارد کردن سیم‌ها به داخل کانکتور RJ-45 تا انتهای پین‌های مس.

۵. پرس کانکتور با آچار سوکت‌زن و تست کابل ساخت‌شده با دستگاه LAN Cable Tester جهت اطمینان از سلامت اتصال تمام ۸ پین.

۵-۳- مهارت کیستون زدن پریز شبکه (Keystone Jack Punch Down)

کیستون ماژولی است که نقطه انتهایی کابل‌های شبکه روکار یا توکار را تشکیل داده و پریز شبکه را ارائه می‌دهد:

شناسایی ساختار کیستون: بررسی پایه‌های اتصال IDC (Insulation Displacement

Connector) و راهنمای رنگی درج‌شده روی بدنه کیستون (برای استانداردهای A و B).

مراحل کیستون‌زنی:

۱. جداسازی روکش کابل به میزان حدود ۳ سانتی‌متر.

۲. قرار دادن هر رشته سیم در شکاف رنگی مربوطه در استاندارد T568B.

۳. استفاده از آچار پانچ / Punch Down Tool (آچار کروز) برای فشردن سیم‌ها به داخل تیغه‌های متصل‌کننده و قطع اضافه سیم به‌صورت همزمان.

۴. بستن درپوش محافظ کیستون و جازدن آن درون فیس‌پلیت (Faceplate) دیوار.

۵-۴- مباحث کلی و کاربردی امنیت شبکه

در طول دوره، مبانی اساسی ایمن‌سازی زیرساخت‌های شبکه نیز آموزش داده شد:

- **مفهوم دفاع در عمق (Defense in Depth):** پیاده‌سازی لایه‌های امنیتی متعدد (فیزیکی، شبکه، سیستم‌عامل، اپلیکیشن) به‌طوری که نفوذ از یک لایه باعث سقوط کل سیستم نشود.
- **دیوار آتش (Firewall):** تنظیم قوانین (Rules) برای کنترل ترافیک ورودی (Inbound) و خروجی (Outbound) بر اساس پورت‌ها و پروتکل‌ها. (Stateful Inspection)
- **کنترل دسترسی (ACLs - Access Control Lists):** ایجاد لیست‌های کنترل دسترسی بر روی روترها برای مسدودسازی IP های ناشناس یا محدود کردن دسترسی به سرورهای حساس.
- **امنیت شبکه‌های بیسیم (Wi-Fi Security):** آشنایی با مکانیزم‌های رمزنگاری WPA^۲/WPA^۳، غیرفعال‌سازی WPS و استفاده از MAC Address Filtering جهت جلوگیری از اتصال دستگاه‌های غیرمجاز.

فصل ۶:

نتیجه‌گیری و پیشنهادات

۶-۱- نتیجه‌گیری

پروژه کارآموزی حاضر با موفقیت کامل به تمامی اهداف تعیین‌شده دست یافت. نرم‌افزار **ArucoDetector** طراحی و پیاده‌سازی گردید که توانست با نرخ فریم بالای ۲۵ فریم بر ثانیه، کدهای ArUco از دیکشنری‌های ۴x۴ و ۶x۶ را به‌صورت زنده، بدون کشیدگی تصویر و بدون هیچگونه Memory Leak شناسایی کند.

علاوه بر بخش نرم‌افزاری، کسب مهارت‌های کارگاهی در زمینه سخت‌افزار شبکه (سوکت‌زنی و کیستون‌زنی)، شبیه‌سازی در Cisco Packet Tracer و درک مفاهیم امنیت شبکه، مجموعه‌ای جامع از دانش عملی را فراهم آورد که مکمل تحصیلات آکادمیک در رشته مهندسی کامپیوتر است.

۶-۲- پیشنهادات برای توسعه آتی

۱. تخمین موقعیت سه‌بعدی (**3D Pose Estimation**): اضافه کردن ماتریس‌های کالیبراسیون دوربین جهت استخراج بردار دوران (rvec) و بردار انتقال (tvec) و رسم مکعب سه‌بعدی مجازی روی مارکر.

۲. اتصال به شبکه و IoT: ارسال شناسه‌های خوانده‌شده از طریق پروتکل MQTT یا HTTP به یک سرور مرکزی یا داشبورد مدیریتی.

۳. توسعه به واقعیت افزوده (AR): نگاشت مدل‌های سه‌بعدی (مانند فرمت‌های OBJ یا GLTF) روی مارکرهای ArUco شناسایی‌شده.

پایان گزارش کارآموزی

تنظیم‌شده توسط: امید شعبانعلی